

```
"""
```

Дерзкий саркастичный Telegram-бот на базе Google Gemini API (бесплатно).

Установка:

```
pip install python-telegram-bot google-generativeai python-dotenv
```

Настройка:

1. Создай файл .env рядом с этим скриптом со строками:

```
TELEGRAM_BOT_TOKEN=твой_токен_от_BotFather
```

```
GEMINI_API_KEY=твой_ключ_с_aistudio.google.com
```

2. Запусти: python bot.py

```
"""
```

```
import os
```

```
import logging
```

```
from collections import defaultdict, deque
```

```
from dotenv import load_dotenv
```

```
from telegram import Update
```

```
from telegram.ext import (
```

```
    Application,
```

```
    CommandHandler,
```

```
    MessageHandler,
```

```
    ContextTypes,
```

```
    filters,
```

```
)
```

```
import google.generativeai as genai
```

```
load_dotenv()
```

```
TELEGRAM_TOKEN = os.getenv("TELEGRAM_BOT_TOKEN")
```

```
GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
```

```
if not TELEGRAM_TOKEN or not GEMINI_API_KEY:
```

```
    raise RuntimeError(
```

```
        "Не найдены TELEGRAM_BOT_TOKEN или GEMINI_API_KEY. "
```

```
        "Проверь файл .env"
```

```
    )
```

```
logging.basicConfig(
```

```
    format="%(asctime)s - %(name)s - %(levelname)s - %(message)s",
```

```
    level=logging.INFO,
```

```
)
```

```
logger = logging.getLogger(__name__)
```

```

genai.configure(api_key=GEMINI_API_KEY)

# Характер бота — можно свободно менять под себя
SYSTEM_PROMPT = """\
Ты — дерзкий, саркастичный собеседник в Telegram-чате. Твой стиль:
- Отвечаешь остроумно, с лёгким подколом, но не переходишь в откровенное хамство.
- Не занудствуешь и не пишешь длинные простыни текста — 1-4 предложения, если не
- Не боишься иронизировать над вопросом собеседника, если он этого заслуживает.
- При этом реально помогаешь и отвечаешь по существу — сарказм это приправа, а не
- Используешь живой разговорный русский язык, без канцелярита.
- Иногда можешь подколоть сам себя.
- Не используешь оскорбления, дискриминацию, токсичность по личным признакам — ёд
"""

model = genai.GenerativeModel(
    model_name="gemini-2.5-flash",
    system_instruction=SYSTEM_PROMPT,
)

# История последних сообщений по каждому чату (чтобы бот помнил контекст)
MAX_HISTORY = 10
chat_histories: dict[int, deque] = defaultdict(lambda: deque(maxlen=MAX_HISTORY))

async def start(update: Update, context: ContextTypes.DEFAULT_TYPE) -> None:
    await update.message.reply_text(
        "Ну привет. Не жди, что я буду сюсюкаться — но раз уж ты тут, давай погов
    )

async def reset(update: Update, context: ContextTypes.DEFAULT_TYPE) -> None:
    chat_histories[update.effective_chat.id].clear()
    await update.message.reply_text("Память чистая, как твои намерения на понедел

async def handle_message(update: Update, context: ContextTypes.DEFAULT_TYPE) -> N
    chat_id = update.effective_chat.id
    user_text = update.message.text

    history = chat_histories[chat_id]

    try:
        # Собираем историю в формате Gemini (role: user/model)
        gemini_history = [
            {"role": "user" if m["role"] == "user" else "model", "parts": [m["con
            for m in history

```

```

    ]
    chat = model.start_chat(history=gemini_history)
    response = chat.send_message(user_text)
    reply_text = response.text.strip()
except Exception as e:
    logger.exception("Ошибка при обращении к Gemini API")
    reply_text = "Что-то я завис. Технические неполадки — не мой день, видимо"

history.append({"role": "user", "content": user_text})
history.append({"role": "model", "content": reply_text})
await update.message.reply_text(reply_text)

def main() -> None:
    app = Application.builder().token(TELEGRAM_TOKEN).build()

    app.add_handler(CommandHandler("start", start))
    app.add_handler(CommandHandler("reset", reset))
    app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, handle_message))

    logger.info("Бот запущен")
    app.run_polling()

if __name__ == "__main__":
    main()

```